

- C# aplikazioa exekutagarri eta instalagarri bihurtzeko prozesua
 - Azaleko fitxa
 - 1. Hasierako egoera eta arazo nagusia
 - 2. dotnet publish bidez exekutagarria sortzea
 - Zergatik ez single-file modua?
 - 3. Konfigurazioa kode barrutik ateratzea
 - 3.1. appsettings.json fitxategia
 - 3.2. AplikazioKonfigurazioaHornitzailea
 - 3.3. AplikazioBideak
 - 4. Datu-basearen konexioa konfiguragarri bihurtzea
 - 5. Bide absolutuak kentzea: PDF, XML eta irudiak
 - 5.1. Erabiltzaileen irudiak
 - 5.2. XML esportazioa
 - 5.3. PDF dokumentuak eta irudien bilaketa
 - 6. Lehen abioa automatizatzea
 - Exekutatzen diren SQL script-ak
 - 7. Inno Setup bidez instalatzailea sortzea
 - Script-eko atal garrantzitsuenak
 - InitializeWizard()
 - CurStepChanged(ssPostInstall)
 - Akats praktiko bat eta konponketa
 - 8. Helburuko ordenagailuan zer egin behar da instalatzeko
 - Gomendio praktikoa
 - 9. Azken emaitza: Setup.exe
 - 10. Laburpena: egin diren moldaketa guztiak
 - 11. Ondorioa
 - Entregarako oharra

C# aplikazioa exekutagarri eta instalagarri bihurtzeko prozesua

Dokumentu honek azaltzen du nola bihurtu den GOSasun aplikazioa garapeneko C# WinForms proiektu hutsetik beste ordenagailu batean instalatu daitekeen exekutagarri batean. Helburua ez zen bakarrik **exe** bat sortzea, baizik eta aplikazioa instalazio errealean martxan geratzeko behar zituen mendekotasun guztiak prest uztea: runtime-

a, datu-basearen konfigurazioa, web/Apache bideak, lehen abioan exekutatu beharreko SQL script-ak eta instalatzaile gidatua.

Azaleko fitxa

- Proiektua: **G0sasun_app**
- Teknologia: **.NET WinForms + MySQL + Inno Setup**
- Helburua: beste Windows ordenagailu batean instalatu daitekeen pakete autoedukia sortzea
- Azken emaitza: **deployment\output\G0sasun_app_Setup.exe**

Jarraian dauden irudiak pantailazo tekniko moduan sortu dira. Prozesuan benetan erabili diren komando, fitxategi eta kode zatietan oinarrituta daude.

1. Hasierako egoera eta arazo nagusia

Hasierako proiektua Visual Studio-n edo **dotnet run** bidez exekutatzeko pentsatuta zegoen, baina ez beste ordenagailu batean instalatzeko. Arazo nagusiak hauek ziren:

- datu-basearen konexioaren balioak kodean bertan zeuden hardcoded
- Apache/web bide absolutuak kodean zeuden jarrita
- lehen exekuzioan ez zegoen egiaztapen edo bootstrap prozesurik
- ez zegoen instalatzaile gidaturik
- publish prozesua ez zegoen dokumentatuta edo automatizatuta

Horregatik, lana lau bloketan banatu zen:

1. exekutagarria sortzea
2. konfigurazioa kode barrutik kanpora ateratzea
3. lehen abioa automatizatzea
4. instalatzaile bat sortzea

2. **dotnet publish** bidez exekutagarria sortzea

Lehen pausoa aplikazioaren argitalpen autoedukitua prestatzea izan zen. Horretarako sortu eta doitu zen [deployment/Publish-GOsasun.ps1](#) script-a.

●●● PowerShell - Publish prozesua

```
PS C:\Ander\Workspace\C\proiektuak\GOsasun_app>
```

```
pwsh -ExecutionPolicy Bypass -File .\deployment\Publish-GOsasun.ps1
```

```
param(Configuration = "Release", Runtime = "win-x64")
```

```
dotnet publish GOsasun_app.csproj -c Release -r win-x64 --self-contained true
```

Zergatik horrela?

- single-file modua kendu zen `Assembly.Location` abisuengatik
- self-contained folder publish mantendu zen beste PC batean runtime-a ez eskatzeko
- emaitza `publish\win-x64` karpetan uzten da

Publish-a osatu da:

C:\Ander\Workspace\C\proiektuak\GOsasun_app\publish\win-x64

Irudi honek exekutagarria sortzeko lehen pausoa erakusten du: .NET aplikazioa karpeta autoedukitu batera argitaratzea.

Script honen helburua hau da:

- proiektua **Release** moduan konpilatzea
- **win-x64** helbururako prestatzea
- **--self-contained true** aukerarekin Windows helburuko makinan .NET SDK edo runtime instalatuta ez egotea onartzea
- emaitza **publish/win-x64** karpetan uztea

Erabilitako komandoa:

```
pwsh -ExecutionPolicy Bypass -File .\deployment\Publish-GOsasun.ps1
```

Zergatik ez single-file modua?

Hasieran publish bakar-fitxategi moduan egitea aztertu zen, baina horrek **Assembly.Location** motako abisuak sortzen zituen. Proiektuak baliabideak eta bideak fitxategi-sistemaren arabera erabiltzen zituenez, fidagarriagoa zen karpeta autoedukitu bat sortzea. Horregatik, azken erabakia hau izan zen:

- **single-file** kendu
- **self-contained folder publish** mantendu

Horrela, **publish/win-x64** karpetan geratu ziren **G0sasun_app.exe**, DLL guztiak, irudiak eta gainontzeko baliabideak.

3. Konfigurazioa kode barrutik ateratzea

Exekutagarria sortzea ez zen nahikoa. Aplikazioak instalazio bakoitzean balio desberdinak behar zituen:

- SQL zerbitzariaren IP edo hostname-a
- portua
- datu-basearen izena
- erabiltzailea eta pasahitza
- dokumentu, XML eta irudien web erroa

Horregatik sortu zen **G0sasun_app/Kontrola/Zerbitzuak/AplikazioKonfigurazioa.cs**.

Konfigurazioa kanpora ateratzea: appsettings.json + AplikazioBideak

AplikazioKonfigurazioa.cs

```
public sealed class DatuBaseKonfigurazioa
{
    string Zerbitzaria = "localhost";
    uint Portua = 3306;
    string DatuBasea = "G0sasun_DB";
    string Erabiltzailea = "root";
    string Pasahitza = "1MG32025";
}
```

AplikazioBideak

```
LortuWebErroa()
LortuDokumentuKarpeta()
LortuPazienteDokumentuKarpeta()
LortuXmlKarpeta()
LortuIrudiKarpeta()
ZiurtatuBiltegitratzeKarpetak()
```

Helburua

Lehen hardcoded zeuden DB eta Apache bideak instalazio bakoitzean konfiguragarri bihurtzea.

appsettings.json

```
{
  "bertsioa": 1,
  "datuBasea": {
    "zerbitzaria": "192.168.1.20",
    "portua": 3306,
    "datuBasea": "G0sasun_DB",
    "erabiltzailea": "root",
    "pasahitza": "*****"
  },
  "biltegitratzea": {
    "webErroa": "C:\\\\Apache24-64\\\\htdocs\\\\G0sasun_web"
  },
  "abioa": {
    "sortuDatuBaseEskemaLehenAbioan": true,
    "kargatuHasierakoDatuakLehenAbioan": true
  }
}
```

Eraitza

Instalatzailerak JSON hau sortzen du, eta aplikazioak hortik irakurtzen du.

Fitxategi horrek hiru ideia garrantzitsu ekarri zituen:

3.1. `appsettings.json` fitxategia

Instalazio bakoitzean sortzen den JSON fitxategi bat definitu zen. Bertan gordetzen dira:

- `datuBasea`
- `biltegitratzea`
- `abioa`

Horri esker, aplikazioaren exekutagarria bera berdina izan daiteke, baina instalazio bakoitzak bere konfigurazioa izan dezake.

3.2. `AplikazioKonfigurazioaHornitzailea`

Klase honek konfigurazioa diskoan irakurri eta idazten du. Bere eginkizunak dira:

- `appsettings.json` existitzen ez bada lehenetsitako konfigurazioa sortzea
- konfigurazioa normalizatzea
- cache txiki bat mantentzea
- fitxategia diskoan gordetzea

3.3. `AplikazioBideak`

Lehen zeuden bide absolutu sakabanatuak helper bakar batera eraman ziren. Horri esker, kodeko beste atalak ez dira gehiago `C:\Apache24-64\htdocs\...` bezalako kate finkoetan oinarritzen.

Helper honek kalkulatzeko ditu:

- dokumentuen karpeta
- pazienteen dokumentuen karpeta
- XML esportazioen karpeta
- irudien karpeta

4. Datu-basearen konexioa konfiguragarri bihurtzea

Hurrengo urratsa izan zen GOsasun_app/Repositorioa/DatuBaseKonexioa.cs egokitzea.

Garapeneko hasierako bertsioan:

- `localhost`
- `3306`
- `GOsasun_DB`
- `root`
- pasahitza

balio guztiak kodean bertan finkatuta zeuden.

Benetako hedapen batean, ordea, ez da jada konfigurazio hori erabiltzen. Eskolako instalazio errealean, aplikazioa zerbitzari zentral honetara konektatzen da:

- SQL zerbitzaria: `192.168.115.163`
- portua: `3306`
- datu-basea: `GOsasun_DB`
- erabiltzailea: `adminRoot`
- pasahitza: `pasahitzZaila1234`

Horretarako, eskolako MySQL zerbitzarian `adminRoot` erabiltzailea sortu da root mailako baimenekin, GOsasun aplikazioak behar duen sarbidea izan dezan.

Orain fitxategi horrek `AplikazioKonfigurazioaHornitzailea`-tik irakurtzen ditu balioak, eta `MySqlConnectionStringBuilder` bidez eraikitzen du konexio-katea. Horrek bi onura ekarri zituen:

1. instalatzaileak erabiltzaileari galdetu diezaioke zerbitzariaren IP-a
2. aplikazioa ez dago garapen-ingurune bakar batera lotuta

Gainera, `LortuKonexioa(bool datuBasearekin = true)` gehitu zen, hasierako bootstrap-ean zerbitzaria probatzeko datu-base zehatzik gabe konektatu ahal izateko.

5. Bide absolutuak kentzea: PDF, XML eta irudiak

Konfigurazioa sortzea ez zen nahikoa. Ondoren, aplikazioak bide horiek benetan erabiltzen zituen lekuak egokitu behar izan ziren.

Horretarako moldatu ziren fitxategi hauek:

- [GOsasun_app/Repositorioa/ErabiltzaileDB.cs](#)
- [GOsasun_app/Kontrola/JarraipenaKontrolatzailea.cs](#)
- [GOsasun_app/Kontrola/Zerbitzuak/DokumentuPdfZerbitzua.cs](#)

Instalagarritasuna lortzeko ukitu diren fitxategiak

Deployment eta packaging

deployment/Publish-GOsasun.ps1	dotnet publish exekutatu eta publish\win-x64 sortu
deployment/GOsasun_app.iss	instalatzailea, wizard-a eta appsettings.json sortzea
deployment/INSTALL_USB.md	azken banaketa gida

Runtime konfigurazioa

GOsasun_app/Kontrola/Zerbitzuak/AplikazioKonfigurazioa.cs	DB, web erroa eta lehen-abio aukerak irakurri
GOsasun_app/Repositorioa/DatuBaseKonexioa.cs	konexio katea hardcoded izatetik JSON bidez sortzera pasa
GOsasun_app/Kontrola/Zerbitzuak/HasierakoPrestaketaZerbitzua.cs	karpeta sortu, SQL exekutatu eta konektibitatea probatu
GOsasun_app/Program.cs	login aurretik prestaketa osoa exekutatu

Bide absolutuen ordez centralizatutako bideak

GOsasun_app/Repositorioa/ErabiltzaileDB.cs	erabiltzailearen irudiak web erro berrian gorde
GOsasun_app/Kontrola/JarraipenaKontrolatzailea.cs	XML esportazioa karpeta konfiguragarrian utzi
GOsasun_app/Kontrola/Zerbitzuak/DokumentuPdfZerbitzua.cs	PDF eta irudi bilaketak bide helperretik pasatu

Hau izan da gakoa: aplikazioa ez zen soilik "konpilatu"; lehenik ingurunearekiko mendekotasunak kanpora atera behar izan ziren.

5.1. Erabiltzaileen irudiak

Erabiltzaileen argazkiak gordetzeko logika

`AplikazioBideak.LortuIrudiHelmugaBidea(...)` metodoaren mende utzi zen.

Horrek esan nahi du helmugako karpeta instalazioaren web erroaren arabera kalkulatzen dela.

5.2. XML esportazioa

Jarraipenaren XML esportazioa lehen bide absolutu batera joaten zen. Orain

`AplikazioBideak.LortuXmlKarpeta()` erabiltzen da, eta karpeta ez badago automatikoki sortzen da.

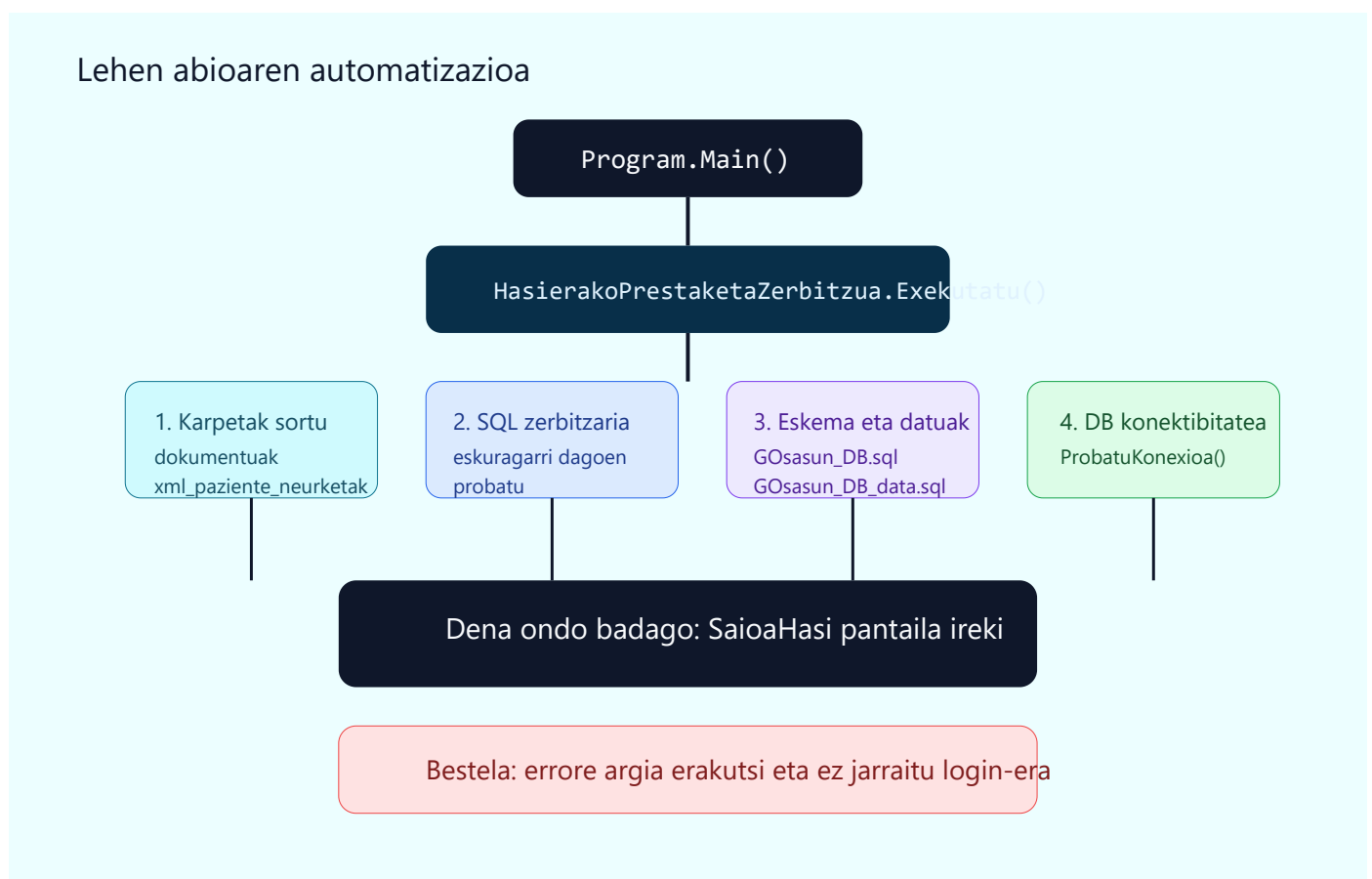
5.3. PDF dokumentuak eta irudien bilaketa

PDF zerbitzuak pazienteen dokumentuak eta irudiak bilatzeko erabiltzen zituen erroak ere helper zentralizatuetera pasa ziren. Horrela, instalatutako aplikazioak ez du garapeneko irudi-erro jakin baten beharrik.

6. Lehen abioa automatizatzea

Instalatutako aplikazio batek ezin du suposatu ingurunea dagoeneko prestatuta dagoela. Horregatik sortu zen

[GOsasun_app/Kontrola/Zerbitzuak/HasierakoPrestaketaZerbitzua.cs](#), eta [GOsasun_app/Program.cs](#) fitxategian login pantaila ireki aurretik deitu zen.



Prozesu hau exekutatzen da aplikazioa lehen aldiz irekitzean:

1. konfigurazioa irakurri
2. web/Apache karpetak existitzen direla ziurtatu
3. SQL zerbitzaria eskuragarri dagoela probatu
4. beharrezkoa bada eskema sortu edo eguneratu
5. beharrezkoa bada seed datuak kargatu
6. azken DB konexio-proba egin
7. dena ondo badago bakarrik **SaioaHasi** pantaila ireki

Exekutatzeko diren SQL script-ak

Bootstrap zerbitzuak `sql` karpetatik script hauek erabil ditzake:

- `sql/GOsasun_DB.sql`
- `sql/GOsasun_DB_trigger.sql`
- `sql/GOsasun_DB_bistak.sql`
- `sql/GOsasun_DB_indizeak.sql`
- `sql/GOsasun_DB_data.sql`

Horrela, aplikazioa lehen exekuzioan bakarrik ez da pizten: bere ingurunea prest uzten du, eta errorea badago erabiltzaileari mezua ematen dio zuzenean.

7. Inno Setup bidez instalatzailea sortzea

Exekutagarria edukita eta runtime konfigurazioa eginda, hurrengo pausoa instalatzaile klasiko bat eraikitzea izan zen. Horretarako doitu zen `deployment/GOsasun_app.iss`.

Inno Setup script-aren moldaketak

`GOsasun_app.iss`

```
DefaultDirName=C:\GOsasun_app
PrivilegesRequired=admin
[Files]
  publish\win-x64 -> {app}
  sql\* -> {app}\sql

InitializeWizard()
  CreateInputDirPage(... web erroa ...)
  CreateInputQueryPage(... SQL host/IP ...)
  CreateInputOptionPage(... eskema/seed ...)

CurStepChanged(ssPostInstall)
  ForceDirectories(...)
  SaveStringToFile({app}\appsettings.json, ...)
```

Script-ak instalazioan galdetzen duena

instalazio karpeta, web erroa, DB host/IP, portua, DB izena, erabiltzailea, pasahitza, eta lehen abioan eskema eta hasierako datuak kargatu nahi diren.

Wizard pantailen laburpena

1. Biltegitratze bideak

Apache/web erroa hautatu

2. Datu-basearen konfigurazioa

SQL zerbitzaria / IP-a
Portua, DB izena, erabiltzailea, pasahitza

3. Lehen exekuzioko prestaketa

Eskema sortu edo eguneratu
Hasierako erregistroak kargatu

Instalatzaileak konfigurazio hori gordetzen du eta aplikazioa prest

Script horrek hainbat eginkizun ditu:

- `publish\win-x64` karpetako edukia `{app}` helburura kopiatu
- `sql` karpetako script-ak instalazioan sartu
- instalazio direktorio lehenetsia proposatu: `C:\G0sasun_app`
- instalazioan zehar galdetu zer web erro erabili behar den
- SQL zerbitzariaren datuak eskatu
- lehen abioan eskema eta seed datuak exekutatu behar diren aukeratu
- amaieran `{app}\appsettings.json` sortu

Gainera, erabiltzailea ez galtzeko, instalazioaren hasieran azalpen-orri bat gehitu zen. Orrialde horrek esaten du zer datu prest izan behar diren:

- instalazio karpeta
- web/Apache erroaren bidea
- SQL zerbitzariaren IP edo hostname-a
- SQL portua, erabiltzailea eta pasahitza
- lehen abioan eskema eta seed datuak kargatu nahi diren ala ez

Eskolako instalazioan, wizard-ean sartu diren datuak hauek izan dira:

- SQL host/IP: `192.168.115.163`
- SQL portua: `3306`
- datu-basea: `G0sasun_DB`
- erabiltzailea: `adminRoot`
- pasahitza: `pasahitzZaila1234`

Script-eko atal garrantzitsuenak

`InitializeWizard()`

Hemen definitzen dira wizard-eko orriak:

- azalpen orri bat instalazioa hasi aurretik
- `CreateInputDirPage` web erroarentzat
- `CreateInputQueryPage` datu-basearen balioentzat
- `CreateInputOptionPage` lehen abio aukerentzat

`CurStepChanged(ssPostInstall)`

Instalazioa amaitutakoan:

- helmugako karpetak sortzen ditu
- `appsettings.json` gordetzen du

Horrek lotzen ditu instalazioan erabiltzaileak emandako balioak eta exekutatzeko aplikazioak irakurriko duen konfigurazioa.

Akats praktiko bat eta konponketa

Instalatzaileraren bertsio zahar batek exekuzioan `publish\win-x64` karpeta bilatzen zuen, eta horrek helburuko ordenagailuan errore hau eragiten zuen:

`Ez da publish karpeta aurkitu. Lehenik exekutatu deployment\Publish-GOsasun.ps1 script-a.`

Hori ez zen zuzena, `Publish-GOsasun.ps1` script-a garapeneko makinan bakarrik exekutatu behar delako, ez instalazioa jasotzen duen ordenagailuan. Horregatik egin zen azken zuzenketa hau:

- runtime-ko egiaztapen hori kendu
- azalpen orri argi bat gehitu instalatzaileraren hasieran

Ondorioz, erabiltzaile arruntak `Setup.exe` exekutatu besterik ez du egin behar.

8. Helburuko ordenagailuan zer egin behar da instalatzeko

Atal hau da erabiltzaile finalak jarraitu behar duena.

1. USBa edo instalatzailea duen karpeta ireki.
2. Exekutatu `deployment/output/GOsasun_app_Setup.exe` administratzaile gisa.
3. Irakurri lehen azalpen-orria eta prest eduki SQL eta web erroaren datuak.
4. Aukeratu instalazio karpeta.
5. Idatzi web/Apache erroaren bidea.
6. Idatzi SQL host/IP-a, portua, datu-basearen izena, erabiltzailea eta pasahitza.
7. Ingurune berria bada, aktibatu lehen abioan eskema eta hasierako datuak kargatzeko aukerak.
8. Amaitu instalazioa eta ireki aplikazioa.

9. Lehen abioan, aplikazioak karpetak sortu, SQL script-ak exekutatu eta DB konexioa egiaztatuko ditu.

Eskolako konfigurazio praktikoan, 6. urratsean balio hauek sartu dira:

- host/IP: 192.168.115.163
- portua: 3306
- datu-basea: G0sasun_DB
- erabiltzailea: adminRoot
- pasahitza: pasahitzZaila1234

Gomendio praktikoa

Instalazioa ingurune berri batean egiten bada, normalean aukerarik seguruenak hau da:

- eskema sortu edo eguneratu: Bai
- hasierako datuak kargatu: Bai

Hala ere, zerbitzari zentral bera dagoeneko prestatuta badago eta beste ordenagailu batean bezeroa bakarrik instalatzen ari bazara, normalean zuzena izango da:

- eskema sortu edo eguneratu: Ez
- hasierako datuak kargatu: Ez

Horrek saihesten du jada existitzen den G0sasun_DB eskema edo datuak berriro kargatzea.

9. Azken emaitza: Setup.exe

Inno Setup konpilatzean lortzen den azken fitxategia hau da:

- [deployment/output/G0sasun_app_Setup.exe](#)

Konpilazio kate osoa honela geratu zen:

Hasierako C# proiektutik installer-era: azken fluxua



Target PC-an gertatzen dena

1. Erabiltzaileak GOSasun_app_Setup.exe exekutatzen du.
2. Instalatzaileak publish karpetako fitxategiak kopiatzen ditu {app}...
3. appsettings.json sortzen du instalazioan emandako balioekin.
4. Lehen abioan, aplikazioak karpetak sortu, SQL scriptak exekutatzen ditu.
5. Dena ondo badoa, erabiltzailea login pantailara iristen da ingurune prestatu batekin.

Azken emaitza

PC batean instalatu daitekeen
GOSasun_app_Setup.exe

Fluxu honek erakusten du transformazioa ez dela build hutsa: konfigurazioa, bootstrap-a eta instalatzailea behar izan dira.

Praktikan, azken banaketa honela egiten da:

1. **Publish-GOSasun.ps1** exekutatu
2. **GOSasun_app.iss** konpilatu **ISCC.exe**-rekin
3. **GOSasun_app_Setup.exe** sortu
4. instalatzailea USB edo beste banaketa-bide batera kopiatu

10. Laburpena: egin diren moldaketa guztiak

Transformazio hau lortzeko ez da nahikoa izan proiektua konpilatzea. Egin diren moldaketa nagusiak hauek izan dira:

1. publish prozesua automatizatu
2. single-file modua baztertu eta karpeta autoedukitua erabili
3. DB eta biltegitratze bideak **appsettings.json**-era atera
4. bide absolutu sakabanatuak helper zentral batean batu
5. datu-basearen konexioa konfiguragarri bihurtu
6. lehen abioan bootstrap automatikoa gehitu
7. SQL eskema eta hasierako datuak exekutatzeke aukera gehitu
8. Inno Setup instalatzaile gidatua prestatu

9. instalazio ondoren `appsettings.json` automatikoki sortzea gehitu

11. Ondorioa

Beraz, C# WinForms proiektu hau exekutagarri eta instalagarri bihurtzeko egin den benetako lana hiru mailatan ulertu behar da:

- build mailan: `dotnet publish`
- konfigurazio mailan: `appsettings.json` eta bide helper-ak
- deployment mailan: Inno Setup + lehen-abio bootstrap-a

Horri esker, gaur egun GOsasun aplikazioa ez da garapeneko makina bakar baterako proiektu bat bakarrik. Beste Windows ordenagailu batean instalatu, konfiguratu eta martxan jar daitekeen aplikazio paketizatua da.

Gainera, dokumentazioa eguneratuta geratzen da benetako eskolako hedapenarekin: aplikazioa ez dago `localhost`-era lotuta, baizik eta `192.168.115.163` zerbitzari zentralera, eta konekzioa `adminRoot` erabiltzailearen bidez egiten da.

Entregarako oharra

PDF batera esportatzerakoan, gomendagarria da dokumentu hau atal bakoitzeko irudiekin batera inprimatzea, irudiak orri-zabalera egokian ipinita eta kode-blokeak monoespazio formatuan mantenduta.